Let’s apply this to the action filter from listing 22.6. Previously, I derived from `ActionFilterAttribute` and obtained an instance of `RecipeService` from the `context` passed to the method. In the following listing I show two classes, `EnsureRecipeExistsFilter` and `EnsureRecipeExistsAttribute`. The filter class is responsible for the functionality and takes in `RecipeService` as a constructor dependency.

Listing 22.11 Using DI in a filter by not deriving from `Attribute`

```
public class EnsureRecipeExistsFilter : IActionFilter                ❶
{
    private readonly RecipeService _service;                         ❷
    public EnsureRecipeExistsFilter(RecipeService service)           ❷
    {                                                                ❷
        _service = service;                                          ❷
    }                                                                ❷
    public void OnActionExecuting(ActionExecutingContext context)    ❸
    {                                                                ❸
        var recipeId = (int) context.ActionArguments["id"];          ❸
        if (!_service.DoesRecipeExist(recipeId))                     ❸
        {                                                            ❸
            context.Result = new NotFoundResult();                   ❸
        }                                                            ❸
    }                                                                ❸

    public void OnActionExecuted(ActionExecutedContext context) { }  ❹
}

public class EnsureRecipeExistsAttribute : TypeFilterAttribute       ❺
{
    public EnsureRecipeExistsAttribute()                             ❻
        : base(typeof(EnsureRecipeExistsFilter)) {}                  ❻
}
```

❶ Doesn’t derive from an Attribute class

❷ RecipeService is injected into the constructor.